



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

- **Uniform-Cost Search (UCS):** UCS only cares about the past. It prioritizes nodes based strictly on the actual cost it took to reach them from the starting position. Using the evaluation function $f(n) = g(n)$, where $g(n)$ is the accumulated path cost, UCS methodically expands outward in all directions like a ripple in a pond. It guarantees the shortest path, but it often wastes a significant amount of computational time exploring dead ends or wrong directions because it has no idea where the goal actually is.
- **A Search:** A* considers both the past and the future. It evaluates nodes using the function $f(n) = g(n) + h(n)$. It keeps the actual path cost $g(n)$, but adds a heuristic estimate, $h(n)$, which guesses the remaining distance to the goal. Because of this heuristic, A* is direction-aware. It actively pulls the search toward the target, aggressively prioritizing paths that appear to be getting closer to the goal while deprioritizing paths that lead away from it.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

- **Why it is admissible:** A heuristic is "admissible" if it *never overestimates* the true mathematical cost to reach a goal. On a 4-way grid, you can only move horizontally and vertically. Manhattan distance calculates the exact number of steps needed assuming a completely empty board. Because walls can only ever *increase* the real step count, the Manhattan estimate will always be equal to or less than the true path cost.

- **The risk of inadmissibility:** If a heuristic overestimates the cost, A* will falsely flag the optimal path as too expensive. It will bypass the best route and settle for a sub-optimal one, completely losing its mathematical guarantee of finding the absolute shortest path.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

- **Admissibility check:** Manhattan distance would **not** be admissible here. If your agent moves diagonally one square up and one square right, that requires 1 step. Manhattan calculates the distance as 2 steps (one up, one right). It overestimates the true cost.
- **The correct metric:** If diagonal and straight moves both cost exactly 1 step, you should switch to **Chebyshev distance** (which takes the maximum of the x-axis difference or y-axis difference). If straight moves cost 1 and diagonal moves reflect real geometry, you should use **Euclidean distance**.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

- Calculating distance to just one food item ignores the layout of the rest of the board, leading to inefficient zigzagging.
- **The Minimum Spanning Tree (MST) Heuristic:** A highly effective, admissible heuristic is to calculate the MST of all remaining food pellets, then add the distance from the agent to the nearest pellet in that tree.
- This estimates the minimum possible steps required to connect all remaining nodes on the board, providing a much tighter, smarter guide for the agent without ever overestimating the true cost.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

Incomplete: {filledCount}/{fields.length} questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING